
Web Application Penetration Test Report — v1.0

Target System: OWASP Juice Shop v19.1.1

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-27

Contents

1	Document Control	2
2	Executive Summary	3
3	Execution of the Security Penetration Test	4
3.1	Subject Under Test	4
3.2	Scope	4
3.3	Methodology	4
3.4	Events	5
4	Risk Assessment	6
4.1	CVSS	6
4.2	Risk Categories	7
4.3	Vulnerability States	8
5	Summary of Findings	9
6	Findings and Remediation	11
6.1	Finding	11
6.1.1	SQL Injection Authentication Bypass on Login Endpoint	11
6.2	Finding	12
6.2.1	Sensitive User Data Exposure via /rest/memories Endpoint	12
6.3	Finding	13
6.3.1	Captcha Answer Leakage in API Response	13
6.4	Finding	14
6.4.1	Prometheus Metrics Exposed Without Authentication	14
6.5	Finding	15
6.5.1	Sensitive File Access via FTP Directory Listing	15
6.6	Finding	16
6.6.1	Security Questions Enumeration via Public API	16
6.7	Finding	17
6.7.1	JWT None Algorithm Authentication Bypass	17
6.8	Finding	18
6.8.1	Default Admin Credentials	18
6.9	Finding	19
6.9.1	Sensitive Data in JWT Token	19

6.10 Finding	19
6.10.1 JWT Public Key Exposure	19
6.11 Finding	20
6.11.1 IDOR - User Data Access via Users API	20
6.12 Finding	21
6.12.1 Premium Access Key Exposure	21
6.13 Finding	22
6.13.1 IDOR - Basket Item Manipulation	22
6.14 Finding	23
6.14.1 Mass Assignment - Privilege Escalation via Admin Role Injection . . .	23
6.15 Finding	24
6.15.1 Deluxe Token Theft and Membership Fraud	24
6.16 Finding	25
6.16.1 Business Logic Flaw - Negative Quantity Basket Items	25
6.17 Finding	26
6.17.1 Stored Cross-Site Scripting (XSS) in Complaints Endpoint	26
7 Appendix A - Lists, Scripts and Raw Data	27
8 Appendix B - List of Abbreviations	28

Scope

OpenHack Security Agent was engaged by Bjoern Kimminich to conduct an external IT security investigation. The subject of the investigation was the “OWASP Juice Shop v19.1.1”.

The test was performed using a local instance of the application at `localhost:3333` in a dedicated test environment..

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent’ conducted the security penetration test on behalf of ‘Bjoern Kimminich’. This report has been prepared exclusively for ‘Bjoern Kimminich’. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered “third parties” within the meaning of this engagement.

‘OpenHack Security Agent’ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-27

Version History

Version	Date	State	Author	Comments
0.1	2026-02-27	Draft	Jane Doe	-
1.0	2026-02-27	Final document	Jane Doe	-

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Bjoern Kimminich	Project Lead	+1 555 345 6789	bjoern.kimminich@owasp.org

2 Executive Summary

OpenHack Security Agent (hereinafter referred to as “ASSESSOR”) has been commissioned by Bjoern Kimminich (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the OWASP Juice Shop v19.1.1 application took place on **2026-02-27**.

For this purpose, the web application, accessible at <http://juiceshop:3000>, was tested from the perspective of an external attacker.

As part of the test, a total of 17 vulnerabilities were identified: 5 critical, 5 high, 7 medium, 0 low, and 0 informational.

Critical: **5** | High: **5** | Medium: **7** | Low: **0** | Informational: **0** | Total: **17**

Table 2.1: Overview of Findings

Severity Count	
Critical	5
High	5
Medium	7
Low	0
Informational	0
Total	17

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

3.3 Methodology

The security penetration test of the OWASP Juice Shop v19.1.1 took place on 2026-02-27.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).

Category	Description
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

DRAFT

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v3.1 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into three groups: “Base”, “Temporal”, and “Environmental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Temporal group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the Base Metrics (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Privileges Required (PR)
- User Interaction (UI)
- Scope (S)
- Confidentiality (C)
- Integrity (I)
- Availability (A)

As penetration testers, we can only assess the general threats posed by a vulnerability through Base Metrics. However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers Environmental Metrics. This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been

submitted before they are remedied. This changes the assessment of the risk and thus the urgency of remediation of a vulnerability. For more information, see [CVSS v3.1 Specification](#).

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v3.1 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active:** The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential:** The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed:** The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
01	SQL Injection Authentication Bypass on Login Endpoint	Critical	9.8
02	Sensitive User Data Exposure via /rest/memories Endpoint	Critical	8.2
03	Captcha Answer Leakage in API Response	High	6.5
04	Prometheus Metrics Exposed Without Authentication	Medium	5.3
05	Sensitive File Access via FTP Directory Listing	Medium	5.3
06	Security Questions Enumeration via Public API	Medium	5.3
07	JWT None Algorithm Authentication Bypass	Critical	9.8
08	Default Admin Credentials	Critical	9.1
09	Sensitive Data in JWT Token	High	7.5
10	JWT Public Key Exposure	Medium	5.3
11	IDOR - User Data Access via Users API	Medium	6.5
12	Premium Access Key Exposure	Medium	6.5
13	IDOR - Basket Item Manipulation	High	8.1
14	Mass Assignment - Privilege Escalation via Admin Role Injection	Critical	9.9
15	Deluxe Token Theft and Membership Fraud	High	7.1

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
16	Business Logic Flaw - Negative Quantity Basket Items	Medium	5.3
17	Stored Cross-Site Scripting (XSS) in Complaints Endpoint	High	7.6

DRAFT

6 Findings and Remediation

6.1 Finding

6.1.1 SQL Injection Authentication Bypass on Login Endpoint

Severity	Critical
CVSS Base Score	9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assets	http://juiceshop:3000/rest/user/login
Status	open

Description

SQL Injection vulnerability in the login endpoint allows complete authentication bypass. An attacker can inject SQL code in the 'email' parameter to bypass authentication and gain access to any account, including the administrator account.

Reproduction Steps: 1. Send a POST request to /rest/user/login with Content-Type: application/json 2. Use the payload: {"email":"' OR 1=1-","password":"anything"} 3. The server returns a valid JWT token for the admin account 4. Alternative payload: {"email":"admin@juice-sh.op' -","password":"x"} also bypasses authentication

Impact: - Complete authentication bypass - Access to all user accounts including administrator - Full database access through SQL injection - No credentials required to gain admin access

Root Cause: The email parameter is directly interpolated into SQL queries without proper parameterization or sanitization.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- SQL Injection Authentication Bypass - Request and Response showing admin JWT token obtained via ' OR 1=1- payload: images/finding-9846cb1b-44a2-4842-9d2b-80d52bb6b225-1-sqli-login-177220282543353238.txt

6.2 Finding

6.2.1 Sensitive User Data Exposure via /rest/memories Endpoint

Severity	Critical
CVSS Base Score	8.2 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N)
Assets	http://juiceshop:3000/rest/memories
Status	open

Description

The /rest/memories endpoint exposes complete user objects including password hashes, authentication tokens, user roles, and other sensitive data without any authentication requirement.

Impact: - Password hashes (MD5) for all users including admin accounts are exposed - Deluxe authentication tokens are exposed, allowing account takeover - User roles are disclosed, revealing admin/deluxe accounts - Email addresses of all users are leaked

Reproduction Steps: 1. Send an unauthenticated GET request to: http://juiceshop:3000/rest/memories
2. The response contains full user objects with: - Password hashes (e.g., '6edd9d726cbdc873c539e41ae8757b8c' for admin) - Deluxe tokens (e.g., 'efe2f1599e2d93440d5243a1ffaf5a413b70cf3ac97156bd6fab9b5ddfcbce0e4') - User roles (admin, deluxe, customer) - Email addresses

Exposed Users: - bjoern.kimminich@gmail.com (admin role) - bjoern@owasp.org (deluxe role) - ethereum@juice-sh.op (deluxe role) - john@juice-sh.op (customer) - emma@juice-sh.op (customer)

Attack Scenario: An attacker can crack the exposed MD5 password hashes offline, or directly use the exposed deluxe tokens to authenticate as deluxe users, gaining elevated privileges without needing credentials.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- JSON response from /rest/memories showing exposed user credentials, password hashes, deluxe tokens, and role information: [images/finding-323bbe98-2856-40ce-a23a-01d3881d1ead-2-memories-user-exposure.json](#)

6.3 Finding

6.3.1 Captcha Answer Leakage in API Response

Severity	High
CVSS Base Score	6.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N)
Assets	http://juiceshop:3000/rest/captcha
Status	open

Description

The /rest/captcha endpoint returns the captcha answer directly in the API response, completely bypassing the security purpose of the captcha mechanism.

Impact: - Captcha security control is completely ineffective - Automated bots can easily bypass captcha protection - Any functionality protected by captcha can be automated

Reproduction Steps: 1. Send a GET request to: http://juiceshop:3000/rest/captcha 2. The response includes the captcha ID, the question, AND the answer: `json { "captchaId": 67, "captcha": "3" }` 3. Use the returned answer in subsequent requests to bypass captcha verification

Attack Scenario: An attacker can automate any action protected by captcha (registration, feedback submission, etc.) by simply extracting the answer from the API response before submitting the form.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- JSON response from /rest/captcha showing the captcha answer '11' returned directly in the API response, bypassing captcha security: [images/finding-3da1bfe7-b5e2-49ad-beba-574691660f6f-3-captcha-answer-leak.json](#)

6.4 Finding

6.4.1 Prometheus Metrics Exposed Without Authentication

Severity	Medium
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/metrics
Status	open

Description

The /metrics endpoint exposes detailed Prometheus metrics without authentication, revealing internal application architecture, performance data, and operational information.

Impact: - Internal application metrics are publicly accessible - Infrastructure information is disclosed - Performance characteristics can be analyzed for attacks - Technology stack details are exposed

Reproduction Steps: 1. Send an unauthenticated GET request to: http://juiceshop:3000/metrics
2. The response contains detailed metrics including: - File upload statistics by file type - Startup duration metrics - CPU and memory usage - Event loop lag statistics - Active resource counts - Process information

Exposed Information: - File upload counts (PDF, executable, images) - Startup task durations - Process memory: ~1.9GB resident, ~3GB virtual - Event loop performance metrics - Active TCP connections and file descriptors

Attack Scenario: An attacker can use this information to understand the application's performance characteristics, identify potential DoS vectors, and gather intelligence about the technology stack for targeted attacks.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Prometheus metrics output from /metrics endpoint exposing internal application metrics, memory usage, CPU stats, and infrastructure details: [images/finding-c74dd890-86d0-41e0-87ee-49d8acdfbb3d-4-prometheus-metrics-exposed.txt](#)

6.5 Finding

6.5.1 Sensitive File Access via FTP Directory Listing

Severity	Medium
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/ftp/ http://juiceshop:3000/ftp/acquisitions.md
Status	open

Description

The /ftp/ endpoint exposes a directory listing containing sensitive files including confidential business documents, order PDFs, and backup files without authentication.

Impact: - Confidential business documents are publicly accessible - Customer order PDFs can be accessed - Backup files with potential credentials are exposed - Internal application files are disclosed

Reproduction Steps: 1. Navigate to: http://juiceshop:3000/ftp/ 2. A full directory listing is displayed showing sensitive files 3. Access individual files directly: - /ftp/acquisitions.md - Confidential acquisition plans - /ftp/legal.md - Legal information - /ftp/coupons_2013.md.bak - Backup file with coupon codes - /ftp/incident-support.kdbx - KeePass database - /ftp/package.json.bak - Application configuration backup - /ftp/order_*.pdf - Customer order documents

Exposed Files: - acquisitions.md (909 bytes) - Confidential acquisition plans marked 'Do not distribute' - legal.md (3047 bytes) - Legal documents - incident-support.kdbx (3246 bytes) - KeePass password database - coupons_2013.md.bak - Old coupon codes - Multiple order PDFs - Customer order documents - package.json.bak, package-lock.json.bak - Application configuration

Attack Scenario: An attacker can download the KeePass database and attempt to crack it offline, access confidential business acquisition plans, or extract coupon codes for financial gain.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Confidential acquisitions.md file accessed via /ftp/ endpoint showing business acquisition plans marked as confidential: [images/finding-d0694c23-28db-4051-a643-fb8725558555-5-acquisitions-confidential.md](#)
- HTML directory listing from /ftp/ endpoint showing all sensitive files including acquisitions.md, incident-support.kdbx (KeePass database), order PDFs, and backup files: [images/finding-d0694c23-28db-4051-a643-fb8725558555-7-ftp-directory-listing.html](#)

6.6 Finding

6.6.1 Security Questions Enumeration via Public API

Severity	Medium
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/api/SecurityQuestions
Status	open

Description

The /api/SecurityQuestions endpoint exposes all security questions used for password recovery without authentication, enabling targeted social engineering attacks.

Impact: - All security questions are publicly enumerable - Attackers can prepare answers for targeted users - Social engineering attack surface is increased - Password reset mechanism is weakened

Reproduction Steps: 1. Send an unauthenticated GET request to: <http://juiceshop:3000/api/SecurityQuestions>
2. The response returns all 14 security questions with IDs: - Your eldest siblings middle name? - Mother's maiden name? - Mother's birth date? (MM/DD/YY) - Father's birth date? (MM/DD/YY) - Maternal grandmother's first name? - Paternal grandmother's first name? - Name of your favorite pet? - Last name of dentist when you were a teenager? - Your ZIP/postal code when you were a teenager? - Company you first work for as an adult? - Your favorite book? - Your favorite movie? - Number of one of your customer or ID cards? - What's your favorite place to go hiking?

Attack Scenario: An attacker targeting a specific user can: 1. Enumerate all possible security questions 2. Research the target to find answers (OSINT) 3. Use the answers to reset the victim's password 4. Take over the account

This vulnerability significantly lowers the bar for account takeover attacks through the password reset functionality.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- JSON response from /api/SecurityQuestions exposing all 14 security questions used for password recovery without authentication: [images/finding-c8beeb0d-9497-4e79-a7b4-360bed8cee4a-6-security-questions-enumeration.json](#)

6.7 Finding

6.7.1 JWT None Algorithm Authentication Bypass

Severity	Critical
CVSS Base Score	9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assets	http://juiceshop:3000/rest/user/login , http://juiceshop:3000/rest/user/whoami , http://juiceshop:3000/rest/user/authentication-details
Status	open

Description

The application accepts JWT tokens with the 'none' algorithm, allowing complete authentication bypass. An attacker can forge a JWT token with 'alg: none' and arbitrary claims (including admin role) without knowing any secret keys. The forged token is accepted by the server and grants full access to authenticated endpoints.

Reproduction Steps: 1. Create a JWT token with header: {"typ":"JWT","alg":"none"}
2. Create a payload with arbitrary user data and role: "admin"
3. Send the token with empty signature (ending with '.')
4. Access protected endpoints like /rest/user/whoami and /rest/user/authentication-details

Impact: - Complete authentication bypass - Access to all 57 user accounts including admin accounts - Exposure of sensitive user data (emails, roles, deluxe tokens, TOTP secrets) - Full administrative access to the application

Evidence: Forged token successfully authenticated and returned user details for attacker@evil.com with admin privileges. The /rest/user/authentication-details endpoint exposed all 57 users in the system.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- JWT None Algorithm Attack - Forged token with alg:none authenticated and exposed all 57 user accounts: [images/finding-c9ba2652-8463-4a51-a4b1-3dfa325f8efe-8-jwt-none-algorithm-attack.txt](#)

6.8 Finding

6.8.1 Default Admin Credentials

Severity	Critical
CVSS Base Score	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	http://juiceshop:3000/rest/user/login
Status	open

Description

The application ships with default admin credentials that are not changed during installation. The default credentials 'admin@juice-sh.op:admin123' allow immediate access to an administrative account.

Reproduction Steps: 1. Navigate to the login endpoint: POST /rest/user/login 2. Submit credentials: {"email":"admin@juice-sh.op","password":"admin123"} 3. Receive valid JWT token with admin role 4. Access administrative functions

Impact: - Unauthorized administrative access to the application - Full control over application data and settings - Ability to view, modify, or delete all user data - Attack surface expansion for further exploitation

Evidence: Login request returned successful authentication with admin role JWT token containing user ID 1 and admin privileges.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Default admin credentials login successful - admin@juice-sh.op:admin123 grants admin JWT token: `images/finding-c6e234f0-b172-489f-8a50-bfda692fa91f-9-default-credentials.txt`

6.9 Finding

6.9.1 Sensitive Data in JWT Token

Severity	High
CVSS Base Score	7.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Assets	http://juiceshop:3000/rest/user/login
Status	open

Description

The JWT token returned by the authentication endpoint contains sensitive user data including the user's password hash. This information should never be included in client-side tokens as it can be easily decoded.

Reproduction Steps: 1. Login to the application via POST /rest/user/login 2. Decode the returned JWT token (base64) 3. Observe the password hash in the payload: "password":"0192023a7bbd73250516f069df18b500"

Impact: - Password hash exposure to any party intercepting the token - Password hash could be cracked offline - Violates principle of least privilege - Sensitive data stored client-side

Evidence: JWT payload contains user data with password hash field visible after base64 decoding. The hash is MD5 format and could be cracked to reveal the actual password.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

6.10 Finding

6.10.1 JWT Public Key Exposure

Severity	Medium
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/encryptionkeys, http://juiceshop:3000/encryptionkeys/jwt.pub
Status	open

Description

The JWT public key is exposed at a publicly accessible URL (/encryptionkeys/jwt.pub). This enables JWT algorithm confusion attacks where an attacker can forge valid tokens by signing with HS256 algorithm using the public key as the HMAC secret.

Reproduction Steps: 1. Access GET /encryptionkeys (directory listing enabled) 2. Download GET /encryptionkeys/jwt.pub 3. Use the public key to perform algorithm confusion attack 4. Create forged JWT with HS256 algorithm signed with the public key

Impact: - Enables JWT forgery via algorithm confusion - Directory listing exposes sensitive files - Combined with other vulnerabilities could lead to authentication bypass

Evidence: RSA public key exposed at /encryptionkeys/jwt.pub. Directory listing at /encryptionkeys shows both jwt.pub and premium.key files.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- JWT public key and premium key exposed in public directory listing: [images/finding-4762fe69-95a3-424e-af0a-6bdf829990ff-10-jwt-pubkey-exposure.txt](#)

6.11 Finding

6.11.1 IDOR - User Data Access via Users API

Severity	Medium
CVSS Base Score	6.5 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)
Assets	http://juiceshop:3000/api/Users/, http://juiceshop:3000/api/Users/{id}

Status	open
--------	------

Description

The `/api/Users/` endpoint allows any authenticated user to list all users and access individual user data by ID, regardless of ownership. A customer role user can view all other users' data including admin accounts.

Reproduction Steps: 1. Login as a customer user (e.g., `jim@juice-sh.op`) 2. Send GET request to `/api/Users/` to list all users 3. Send GET request to `/api/Users/1` to access admin's user data 4. Observe full access to all user information including emails, roles, and deluxe tokens

Evidence: - Customer user 'jim' successfully accessed admin user data (ID=1) - Full user list with 57+ users returned to non-admin user - Deluxe tokens for premium users are exposed

Impact: - Horizontal privilege escalation - any user can view other users' data - User enumeration for targeted attacks - Deluxe token theft for membership fraud

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Customer user accessing full user list including admin accounts and deluxe tokens: `images/finding-5bc35a30-4efd-4853-9ab9-6b65790edc42-12-users-list-idor-1772203242.json`

6.12 Finding

6.12.1 Premium Access Key Exposure

Severity	Medium
CVSS Base Score	6.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N)
Assets	<code>http://juiceshop:3000/encryptionkeys</code> , <code>http://juiceshop:3000/encryptionkeys/premium.key</code>
Status	open

Description

A premium access key is exposed at /encryptionkeys/premium.key. This key (1337133713371337.EA99A61D92D2955B1E9285B55BF2A) may be used to gain premium/deluxe features without authorization.

Reproduction Steps: 1. Access GET /encryptionkeys (directory listing enabled) 2. Download GET /encryptionkeys/premium.key 3. Use the key value: 1337133713371337.EA99A61D92D2955B1E9285B55BF2A

Impact: - Unauthorized access to premium features - Potential revenue loss for the application - Directory listing misconfiguration

Evidence: Premium key exposed at /encryptionkeys/premium.key with value 1337133713371337.EA99A61D92D2955B1E9285B55BF2A

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Premium access key exposed in public directory - 1337133713371337.EA99A61D92D2955B1E9285B55BF2A
images/finding-9157de68-e6ca-41c2-94c3-f736862f9f6b-11-jwt-pubkey-exposure.txt

6.13 Finding

6.13.1 IDOR - Basket Item Manipulation

Severity	High
CVSS Base Score	8.1 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)
Assets	http://juiceshop:3000/api/BasketItems/ http://juiceshop:3000/api/BasketItems/{id}
Status	open

Description

The /api/BasketItems/ endpoint allows any authenticated user to read, modify, and delete other users' basket items by manipulating the basket item ID. This enables cross-user shopping cart manipulation.

Reproduction Steps: 1. Login as customer user 'jim' (ID=2, BasketId=2) 2. Send GET request to /api/BasketItems/ to list all basket items (returns items from all users) 3. Send GET request to /api/BasketItems/1 to access admin's basket item 4. Send PUT request to /api/BasketItems/1 with {"quantity": 5} to modify admin's basket 5. Send DELETE request to /api/BasketItems/1 to delete admin's basket item 6. All operations succeed - admin's basket is compromised

Evidence: - Jim successfully viewed admin's basket item (BasketId=1, ProductId=1) - Jim successfully modified admin's basket item quantity from 2 to 5 - Jim successfully deleted admin's basket item

Impact: - Attackers can manipulate other users' shopping carts - Can add/remove products from victims' baskets - Can cause financial harm by modifying quantities - Can disrupt legitimate orders

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Customer user viewing all basket items from all users including admin basket: [images/finding-00ec06ae-e48d-4cca-8337-77c34f6ba7b0-13-idor-basketitems-1772203076.json](#)

6.14 Finding

6.14.1 Mass Assignment - Privilege Escalation via Admin Role Injection

Severity	Critical
CVSS Base Score	9.9 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H)
Assets	http://juiceshop:3000/api/Users/
Status	open

Description

The /api/Users/ endpoint allows any authenticated user to create new accounts with arbitrary roles, including admin. A customer-role user can create admin accounts by injecting the 'role' parameter in the POST request body.

Reproduction Steps: 1. Login as a customer user (e.g., jim@juice-sh.op) 2. Send POST request to /api/Users/ with JSON body: {"email":"evil@admin.com","password":"hacked123","role":"admin"} 3. Observe successful response with newly created admin user 4. Login with the new admin credentials to confirm admin privileges

Evidence: - Customer user 'jim' (ID=2, role=customer) successfully created admin user - New user created with ID=58, email='evil@admin.com', role='admin' - The 'role' field is accepted without validation against the authenticated user's privileges

Impact: - Complete vertical privilege escalation - Any authenticated user can become admin
- Full system compromise - Unauthorized access to all administrative functions - Potential for data breach and system takeover

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Customer user creating admin account via mass assignment - role parameter injection allows privilege escalation: [images/finding-6271a710-df9b-40ca-881e-14b3fa2450ee-14-mass-assignment-admin-creation-1772203560.txt](#)

6.15 Finding

6.15.1 Deluxe Token Theft and Membership Fraud

Severity	High
CVSS Base Score	7.1 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N)
Assets	http://juiceshop:3000/rest/deluxe-membership , http://juiceshop:3000/rest/memories
Status	open

Description

The deluxe membership token is exposed in the /rest/memories endpoint without authentication, and can be used by any user to fraudulently upgrade their account to deluxe membership status without payment.

Reproduction Steps: 1. Access /rest/memories endpoint (no authentication required) 2. Extract deluxe token from any deluxe user (e.g., 'efe2f1599e2d93440d5243a1ffaf5a413b70cf3ac97156bd6fab9b5ddfd' for bjoern@owasp.org) 3. Login as any customer user 4. Send POST request to /rest/deluxe-membership with stolen deluxeToken 5. Receive 'Congratulations! You are now a deluxe member!' response 6. JWT token is updated with role: 'deluxe'

Impact: - Free deluxe membership (\$49 value) without payment - Loss of revenue for the business - Privilege escalation to deluxe member benefits - Any user can impersonate deluxe members

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Deluxe membership fraud using stolen token from memories endpoint - user upgraded without payment: [images/finding-fe07d3c3-0071-40e9-9311-6666cf572dce-16-deluxe-token-fraud-1772204158.txt](#)

6.16 Finding

6.16.1 Business Logic Flaw - Negative Quantity Basket Items

Severity	Medium
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:L)
Assets	http://juiceshop:3000/api/BasketItems/
Status	open

Description

The [/api/BasketItems/](#) endpoint accepts negative quantity values when adding items to a basket. This allows users to potentially exploit the system by creating negative-priced basket items.

Reproduction Steps: 1. Login as any authenticated user 2. Send POST request to [/api/BasketItems/](#) with quantity: -100 (or any negative value) 3. Observe successful creation with negative quantity

Evidence: - Successfully created basket item with quantity: -100 - Response: {"id":20,"ProductId":1,"BasketId":2,"quantity":100}

Impact: - Potential financial fraud via negative pricing - System manipulation - Could result in refunds or credits being generated fraudulently - Violates business logic constraints

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Basket item created with negative quantity -100 demonstrates business logic flaw: [images/finding-a8035d3d-d593-494d-aeb8-94c9cc4e4300-15-negative-quantity-1772204158.txt](#)

6.17 Finding

6.17.1 Stored Cross-Site Scripting (XSS) in Complaints Endpoint

Severity	High
CVSS Base Score	7.6 (CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N)
Assets	http://juiceshop:3000/api/Complaints
Status	open

Description

Stored Cross-Site Scripting (XSS) vulnerability in the Complaints API allows authenticated users to inject malicious JavaScript that executes when viewed by administrators or other users.

Reproduction Steps: 1. Authenticate with any valid account 2. Send POST request to /api/Complaints with JSON body: {"message": "3. The XSS payload is stored in the database without sanitization 4. When an admin or other user views complaints, the malicious script executes"}

Impact: - Session hijacking by stealing cookies - Credential theft through fake login forms - Malware distribution through drive-by downloads - Phishing attacks within the application - Attackers can impersonate victims or perform actions on their behalf

Root Cause: The complaint message field does not sanitize or escape HTML/JavaScript content before storing it in the database or rendering it in the UI.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Stored XSS in Complaints - Malicious script payload stored without sanitization, executes when viewed by admins: [images/finding-24c47fdb-b2e3-4113-b2fc-48d37da5a768-17-stored-xss-complaint-1772204366191225100.txt](#)

7 Appendix A - Lists, Scripts and Raw Data

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+-----+-----+